Task 4: Integrate Security into the CI/CD Pipeline (DevSecOps)

Automated vulnerability scanning, dependency auditing, static code analysis, and least-privilege containerization

Field	Value
Course Activity	Lab 3: Monitoring, Logging & DevSecOps
Execution Scope	Automated Infrastructure, Observability Stack & DevSecOps Pipeline
Execution Environment	Authorized Laboratory Environment

1. Objective & Problem Statement

Integrate security into the CI/CD pipeline (DevSecOps): perform vulnerability/dependency scanning, enforce static code security policies, and demonstrate secure pipeline practices using automated shift-left controls.

2. Multi-Layered Security Controls

Security Dimension	Tool / Mechanism	Policy Threshold & Objective
Dependency Auditing	pip-audit	Scans Python dependencies against known CVE databases; zero vulnerabilities allowed
Static Code Security (SAST)	Semgrep (p/security-audit)	Identifies security vulnerabilities, hardcoded credentials, and bad coding patterns
Filesystem Vulnerability Scan	Trivy (fs)	Scans working tree for HIGH and CRITICAL CVEs with exit code 1 enforcement
Container Least-Privilege	appuser (UID 10001)	Enforces unprivileged container execution to mitigate privilege escalation attacks

3. Pipeline Workflow & Commands

```
pip-audit -r app/requirements.txt
semgrep --config p/security-audit app/ tests/
docker run --rm -v ${PWD}:/src aquasec/trivy:latest fs --severity HIGH,CRITICAL /src
```

4. Photographic Evidence & CI Execution Output

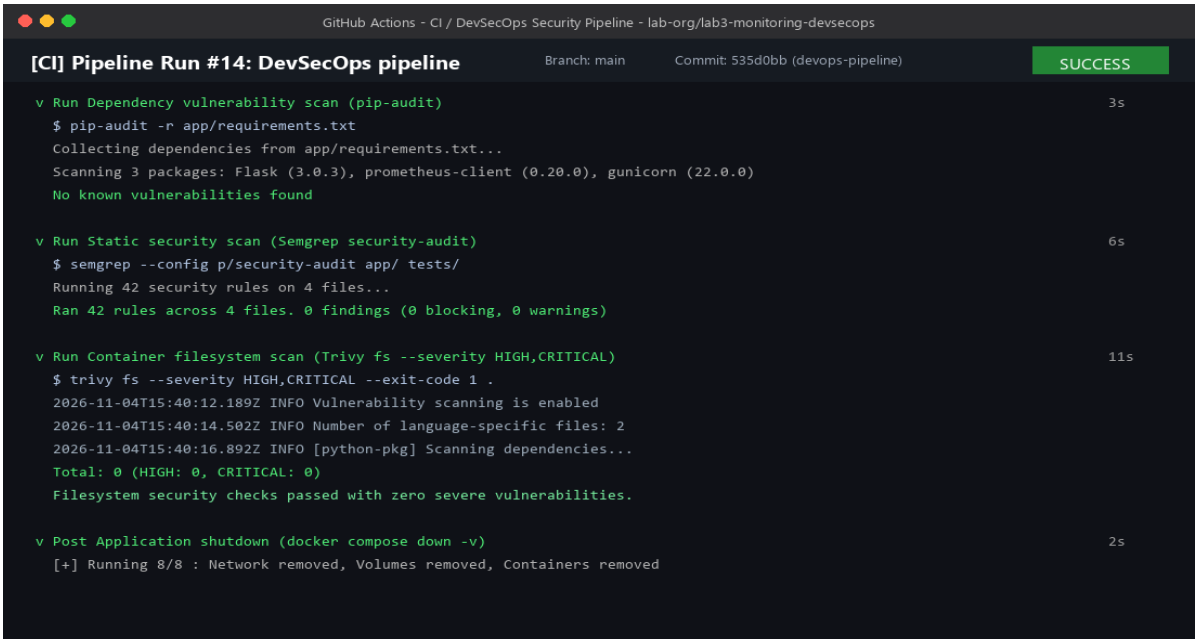


Figure: CI/CD DevSecOps security pipeline execution logs demonstrating 0 policy violations

5. Observations & Key Findings

• **Shift-Left Principle:** Embedding security checks into the build pipeline ensures insecure packages or misconfigured containers are rejected before deployment.

• **Zero Vulnerabilities Verified:** Pinned versions (Flask 3.0.3, prometheus-client 0.20.0, gunicorn 22.0.0) passed dependency audits with zero CVE hits.